**Classical vs Quantum Comparison Tool_ Deutsch-Jozsa Algorithm**

**NAME AAYUSH SUTHAR**

**REG NO 23FE10CAI00275**

**SEC 6 F**

**BATCH 1**

```
!pip install qiskit qiskit-aer pylatexenc
```

```
Requirement already satisfied: qiskit in /usr/local/lib/python3.12/dist-packages (2.3.1)
Requirement already satisfied: qiskit-aer in /usr/local/lib/python3.12/dist-packages (0.17.2)
Requirement already satisfied: pylatexenc in /usr/local/lib/python3.12/dist-packages (2.10)
Requirement already satisfied: rustworkx>=0.15.0 in /usr/local/lib/python3.12/dist-packages (from qiskit) (0.17.1
Requirement already satisfied: numpy<3,>=1.21 in /usr/local/lib/python3.12/dist-packages (from qiskit) (2.0.2)
Requirement already satisfied: scipy>=1.5 in /usr/local/lib/python3.12/dist-packages (from qiskit) (1.16.3)
Requirement already satisfied: dill>=0.3 in /usr/local/lib/python3.12/dist-packages (from qiskit) (0.3.8)
Requirement already satisfied: stevedore>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from qiskit) (5.7.0)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from qiskit) (4.15.0
Requirement already satisfied: psutil>=5 in /usr/local/lib/python3.12/dist-packages (from qiskit-aer) (5.9.5)
Requirement already satisfied: python-dateutil>=2.8.0 in /usr/local/lib/python3.12/dist-packages (from qiskit-aer
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.0->
```

```
import numpy as np
import matplotlib.pyplot as plt
import random
import time
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector
print("All imports successful!")
```

```
All imports successful!
```

```
# ============================================================
# CELL 3: Oracle Construction
# ============================================================
def create_constant_oracle(n, constant_value=0):
    """
    Constant oracle:
    - constant_value=0: f(x) = 0 for all x (do nothing)
    - constant_value=1: f(x) = 1 for all x (flip output qubit)
    """
    oracle = QuantumCircuit(n + 1, name=f"Constant-{constant_value}")
    if constant_value == 1:
        oracle.x(n)
    return oracle


def create_balanced_oracle(n, seed=None):
    """
    Balanced oracle using random CNOT gates.
    At least one input qubit controls the output qubit.
    """
    if seed is not None:
        random.seed(seed)
    oracle = QuantumCircuit(n + 1, name="Balanced")
    control_qubits = list(range(n))
    random.shuffle(control_qubits)
    num_controls = random.randint(1, n)
    selected_controls = control_qubits[:num_controls]
    for ctrl in selected_controls:
        oracle.cx(ctrl, n)
    return oracle


print("Oracle functions defined.")
```

```
Oracle functions defined.
```

```
# ============================================================
# CELL 4: Classical Approach
# ============================================================
def classical_oracle_function(x_binary, oracle_type, n, balanced_controls=None):
    """Simulate classical black-box evaluation of f(x)."""
    if oracle_type == "constant_0":
        return 0
```

```
    elif oracle_type == "constant_1":
        return 1
    else:
        result = 0
        for ctrl in balanced_controls:
            result ^= int(x_binary[ctrl])
        return result


def classical_deutsch_jozsa(n, oracle_type, balanced_controls=None):
    """
    Classical deterministic algorithm.
    Worst case: 2^(n-1) + 1 evaluations.
    Returns (result, num_evaluations, evaluation_log).
    """
    total_inputs = 2 ** n
    evaluations = 0
    log = []
    first_output = None

    for i in range(total_inputs):
        x_binary = format(i, f'0{n}b')
        output = classical_oracle_function(x_binary, oracle_type, n, balanced_controls)
        evaluations += 1
        log.append((x_binary, output))

        if first_output is None:
            first_output = output
        elif output != first_output:
            return "balanced", evaluations, log

        if evaluations > 2 ** (n - 1):
            return "constant", evaluations, log

    return "constant", evaluations, log


# Quick demo
n_demo = 3
result, evals, log = classical_deutsch_jozsa(n_demo, "balanced", [0, 2])
print(f"Classical check for n={n_demo} (balanced oracle):")
print(f"  Result: {result}")
print(f"  Evaluations needed: {evals}")
print(f"  Worst case: {2**(n_demo-1)+1}")
print(f"\n  Evaluation log:")
for x, fx in log:
    print(f"    f({x}) = {fx}")
```

```
Classical check for n=3 (balanced oracle):
  Result: balanced
  Evaluations needed: 2
  Worst case: 5

  Evaluation log:
    f(000) = 0
    f(001) = 1
```

```
# ============================================================
# CELL 5: Quantum Approach - Deutsch-Jozsa Algorithm
# ============================================================
def deutsch_jozsa_circuit(n, oracle):
    """
    Build the full Deutsch-Jozsa circuit.
    Steps:
      1. Init output qubit to |1>
      2. Hadamard on all qubits
      3. Apply oracle
      4. Hadamard on input qubits
      5. Measure input qubits
    """
    qc = QuantumCircuit(n + 1, n)

    # Step 1: output qubit |1>
    qc.x(n)

    # Step 2: Hadamard all
    qc.h(range(n + 1))
    qc.barrier()

    # Step 3: Oracle
    qc.compose(oracle, inplace=True)
    qc.barrier()
```

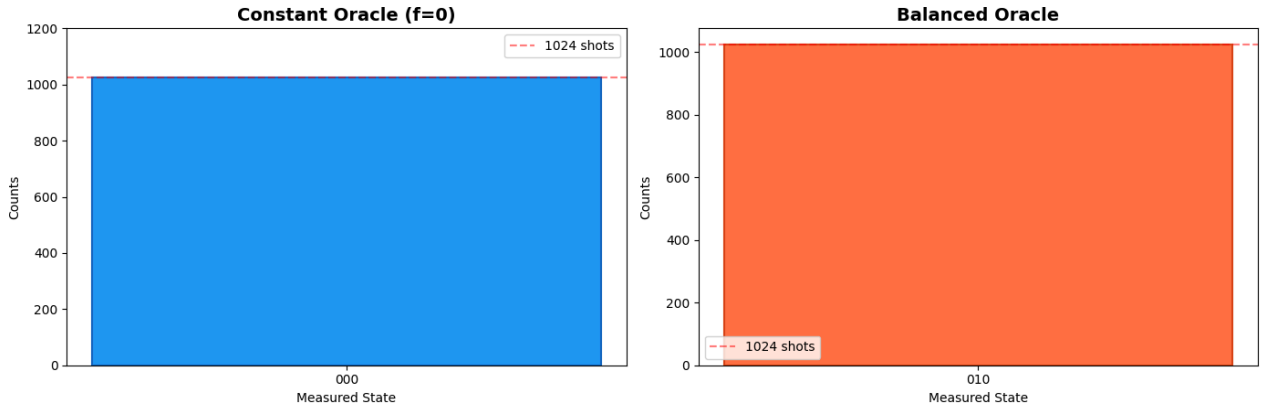
```
    # Step 4: Hadamard input qubits
    qc.h(range(n))

    # Step 5: Measure
    qc.measure(range(n), range(n))

    return qc


def run_deutsch_jozsa(n, oracle, shots=1024):
    """Run the algorithm and return (result, counts, circuit)."""
    qc = deutsch_jozsa_circuit(n, oracle)
    simulator = AerSimulator()
    compiled = transpile(qc, simulator)
    job = simulator.run(compiled, shots=shots)
    counts = job.result().get_counts()

    zero_state = '0' * n
    if zero_state in counts and counts[zero_state] == shots:
        result = "constant"
    else:
        result = "balanced"

    return result, counts, qc


print("Deutsch-Jozsa functions defined.")
```

```
Deutsch-Jozsa functions defined.
```

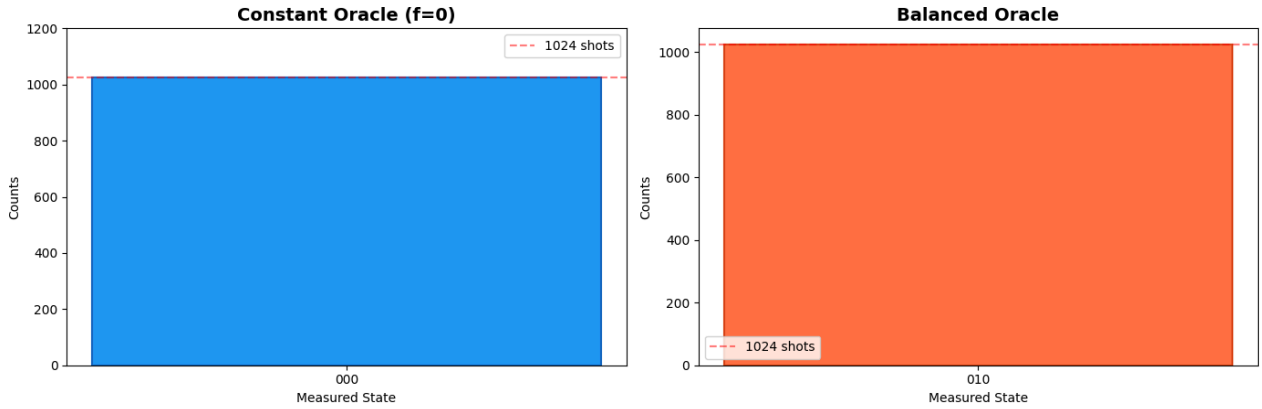
```
# ============================================================
# CELL 6: Side-by-Side Demo
# ============================================================
n = 3

print("=" * 60)
print(f"  DEUTSCH-JOZSA COMPARISON  —  n = {n} qubits")
print(f"  Total possible inputs: {2**n}")
print("=" * 60)

# --- Constant Oracle ---
print("\n" + "—" * 60)
print("TEST 1: Constant-0 Oracle  f(x) = 0 for all x")
print("—" * 60)

result_c, evals_c, _ = classical_deutsch_jozsa(n, "constant_0")
print(f"\n  Classical:  Result = {result_c},  Evaluations = {evals_c}")

oracle_const = create_constant_oracle(n, 0)
result_q, counts_q, _ = run_deutsch_jozsa(n, oracle_const)
print(f"  Quantum:    Result = {result_q},  Evaluations = 1")
print(f"  Measurement counts: {counts_q}")

# --- Balanced Oracle ---
print("\n" + "—" * 60)
print("TEST 2: Balanced Oracle")
print("—" * 60)

result_c, evals_c, _ = classical_deutsch_jozsa(n, "balanced", [0, 1])
print(f"\n  Classical:  Result = {result_c},  Evaluations = {evals_c}")

oracle_bal = create_balanced_oracle(n, seed=42)
result_q, counts_q, _ = run_deutsch_jozsa(n, oracle_bal)
print(f"  Quantum:    Result = {result_q},  Evaluations = 1")
print(f"  Measurement counts: {counts_q}")

# ============================================================
# CELL 7: Visualize Quantum Circuit
# ============================================================
n_vis = 3
oracle_vis = create_balanced_oracle(n_vis, seed=42)
qc_vis = deutsch_jozsa_circuit(n_vis, oracle_vis)

print(f"Deutsch-Jozsa Circuit for n={n_vis}:\n")
qc_vis.draw(output='mpl', style='iqp', fold=-1)
plt.tight_layout()
plt.show()
```

```
============================================================
  DEUTSCH-JOZSA COMPARISON  —  n = 3 qubits
  Total possible inputs: 8
============================================================

────────────────────────────────────────────────────────────
TEST 1: Constant-0 Oracle  f(x) = 0 for all x
────────────────────────────────────────────────────────────

  Classical:  Result = constant,  Evaluations = 5
  Quantum:    Result = constant,  Evaluations = 1
  Measurement counts: {'000': 1024}

────────────────────────────────────────────────────────────
TEST 2: Balanced Oracle
────────────────────────────────────────────────────────────

  Classical:  Result = balanced,  Evaluations = 3
  Quantum:    Result = balanced,  Evaluations = 1
  Measurement counts: {'010': 1024}
Deutsch-Jozsa Circuit for n=3:

<Figure size 640x480 with 0 Axes>
```

```
# ============================================================
# CELL 8: Measurement Histograms
# ============================================================
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Constant oracle
oracle_c0 = create_constant_oracle(3, 0)
_, counts_c0, _ = run_deutsch_jozsa(3, oracle_c0, shots=1024)

# Balanced oracle
oracle_b = create_balanced_oracle(3, seed=42)
_, counts_b, _ = run_deutsch_jozsa(3, oracle_b, shots=1024)

# Plot constant
states_c0 = list(counts_c0.keys())
vals_c0 = list(counts_c0.values())
axes[0].bar(states_c0, vals_c0, color='#2196F3', edgecolor='#1565C0', linewidth=1.5)
axes[0].set_title('Constant Oracle (f=0)', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Measured State')
axes[0].set_ylabel('Counts')
axes[0].set_ylim(0, 1200)
axes[0].axhline(y=1024, color='red', linestyle='--', alpha=0.5, label='1024 shots')
axes[0].legend()

# Plot balanced
states_b = list(counts_b.keys())
vals_b = list(counts_b.values())
axes[1].bar(states_b, vals_b, color='#FF7043', edgecolor='#D84315', linewidth=1.5)
axes[1].set_title('Balanced Oracle', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Measured State')
axes[1].set_ylabel('Counts')
axes[1].axhline(y=1024, color='red', linestyle='--', alpha=0.5, label='1024 shots')
axes[1].legend()

plt.suptitle('Deutsch-Jozsa Measurement Results (n=3)', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```



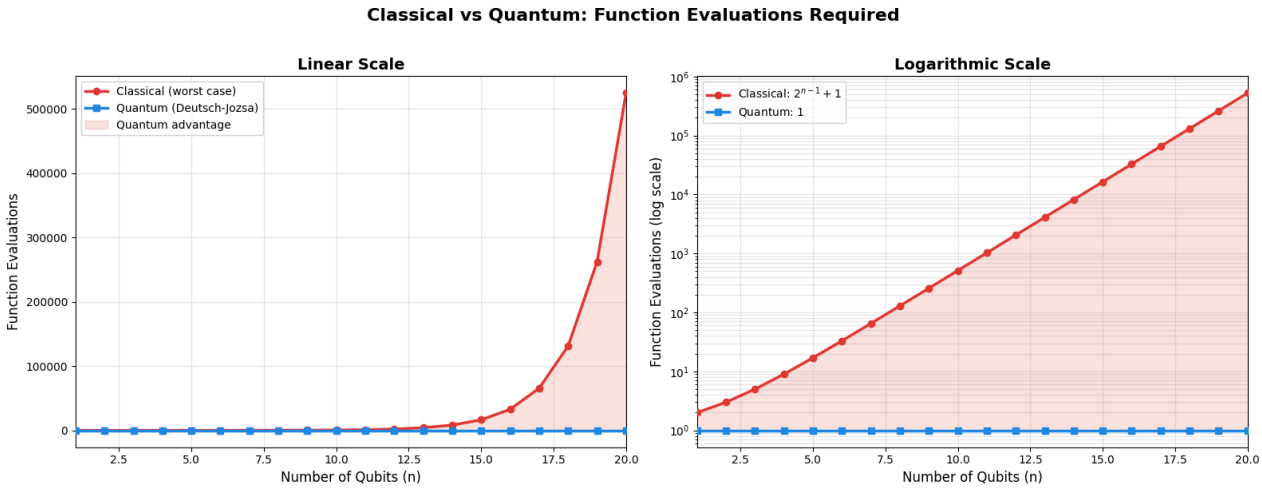

```
# ============================================================
# CELL 8: Measurement Histograms
# ============================================================
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Constant oracle
oracle_c0 = create_constant_oracle(3, 0)
_, counts_c0, _ = run_deutsch_jozsa(3, oracle_c0, shots=1024)

# Balanced oracle
oracle_b = create_balanced_oracle(3, seed=42)
_, counts_b, _ = run_deutsch_jozsa(3, oracle_b, shots=1024)

# Plot constant
states_c0 = list(counts_c0.keys())
vals_c0 = list(counts_c0.values())
axes[0].bar(states_c0, vals_c0, color='#2196F3', edgecolor='#1565C0', linewidth=1.5)
axes[0].set_title('Constant Oracle (f=0)', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Measured State')
axes[0].set_ylabel('Counts')
axes[0].set_ylim(0, 1200)
axes[0].axhline(y=1024, color='red', linestyle='--', alpha=0.5, label='1024 shots')
axes[0].legend()

# Plot balanced
states_b = list(counts_b.keys())
vals_b = list(counts_b.values())
axes[1].bar(states_b, vals_b, color='#FF7043', edgecolor='#D84315', linewidth=1.5)
axes[1].set_title('Balanced Oracle', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Measured State')
axes[1].set_ylabel('Counts')
axes[1].axhline(y=1024, color='red', linestyle='--', alpha=0.5, label='1024 shots')
axes[1].legend()

plt.suptitle('Deutsch-Jozsa Measurement Results (n=3)', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

print("\n-> Constant oracle: ALL measurements collapse to |000>")
print("-> Balanced oracle: measurements are NON-ZERO states (never |000>)")
```




```
-> Constant oracle: ALL measurements collapse to |000>
-> Balanced oracle: measurements are NON-ZERO states (never |000>)
```

```
# ============================================================
# CELL 9: ⭐ Exponential Growth Comparison Plot
# ============================================================
def plot_complexity_comparison(max_n=20):
    """Plot classical vs quantum query complexity."""
    n_values = np.arange(1, max_n + 1)
    classical_evals = 2 ** (n_values - 1) + 1
    quantum_evals = np.ones_like(n_values)

    fig, axes = plt.subplots(1, 2, figsize=(16, 6))

    # Linear Scale
    ax1 = axes[0]
    ax1.plot(n_values, classical_evals, 'o-', color='#E53935', linewidth=2.5,
             markersize=6, label='Classical (worst case)', zorder=5)
    ax1.plot(n_values, quantum_evals, 's-', color='#1E88E5', linewidth=2.5,
             markersize=6, label='Quantum (Deutsch-Jozsa)', zorder=5)
    ax1.fill_between(n_values, quantum_evals, classical_evals,
                     alpha=0.15, color='#E53935', label='Quantum advantage')
    ax1.set_xlabel('Number of Qubits (n)', fontsize=12)
    ax1.set_ylabel('Function Evaluations', fontsize=12)
    ax1.set_title('Linear Scale', fontsize=14, fontweight='bold')
    ax1.legend(fontsize=10)
    ax1.grid(True, alpha=0.3)
    ax1.set_xlim(1, max_n)

    # Log Scale
    ax2 = axes[1]
    ax2.semilogy(n_values, classical_evals, 'o-', color='#E53935', linewidth=2.5,
                 markersize=6, label='Classical: $2^{n-1}+1$', zorder=5)
    ax2.semilogy(n_values, quantum_evals, 's-', color='#1E88E5', linewidth=2.5,
                 markersize=6, label='Quantum: $1$', zorder=5)
    ax2.fill_between(n_values, quantum_evals, classical_evals, alpha=0.15, color='#E53935')
    ax2.set_xlabel('Number of Qubits (n)', fontsize=12)
    ax2.set_ylabel('Function Evaluations (log scale)', fontsize=12)
    ax2.set_title('Logarithmic Scale', fontsize=14, fontweight='bold')
    ax2.legend(fontsize=10)
    ax2.grid(True, alpha=0.3, which='both')
    ax2.set_xlim(1, max_n)

    plt.suptitle('Classical vs Quantum: Function Evaluations Required',
                 fontsize=16, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.show()

    # Print table
    print("\n" + "=" * 55)
    print(f"{'n':>4} | {'Classical (2^(n-1)+1)':>22} | {'Quantum':>8} | {'Speedup':>12}")
    print("-" * 55)
    for i, nv in enumerate(n_values):
        speedup = classical_evals[i] / quantum_evals[i]
        print(f"{nv:>4} | {classical_evals[i]:>22,} | {quantum_evals[i]:>8} | {speedup:>11,.0f}x")
```

```
plot_complexity_comparison(max_n=20)
```




```
========================================================
   n |  Classical (2^(n-1)+1) |  Quantum |      Speedup
--------------------------------------------------------
   1 |                      2 |        1 |           2x
   2 |                      3 |        1 |           3x
   3 |                      5 |        1 |           5x
   4 |                      9 |        1 |           9x
   5 |                     17 |        1 |          17x
   6 |                     33 |        1 |          33x
   7 |                     65 |        1 |          65x
   8 |                    129 |        1 |         129x
   9 |                    257 |        1 |         257x
  10 |                    513 |        1 |         513x
  11 |                  1,025 |        1 |       1,025x
  12 |                  2,049 |        1 |       2,049x
  13 |                  4,097 |        1 |       4,097x
  14 |                  8,193 |        1 |       8,193x
  15 |                 16,385 |        1 |      16,385x
  16 |                 32,769 |        1 |      32,769x
  17 |                 65,537 |        1 |      65,537x
  18 |                131,073 |        1 |     131,073x
  19 |                262,145 |        1 |     262,145x
  20 |                524,289 |        1 |     524,289x
```

```
# ============================================================
# CELL 10: 🎮 Interactive Tool - User Input
# ============================================================
def interactive_comparison(n, oracle_type="random", seed=None):
    """
    Full interactive comparison for user-specified n.

    Parameters:
        n: Number of input qubits (1-15 for simulation)
        oracle_type: 'constant_0', 'constant_1', 'balanced', or 'random'
        seed: Random seed for reproducibility
    """
    if seed is not None:
        random.seed(seed)

    print("\n" + "═" * 60)
    print(f"  ⚛️  DEUTSCH-JOZSA: CLASSICAL vs QUANTUM")
    print(f"  Input qubits: n = {n}")
    print(f"  Total input space: 2^{n} = {2**n:,} possible inputs")
    print("═" * 60)

    # Determine oracle
    if oracle_type == "random":
        choice = random.choice(["constant_0", "constant_1", "balanced"])
    else:
        choice = oracle_type

    if choice in ["constant_0", "constant_1"]:
        balanced_controls = None
```

```
else:
    controls = list(range(n))
    random.shuffle(controls)
    balanced_controls = sorted(controls[:random.randint(1, n)])

print(f"\n  Oracle type: {choice}")
if balanced_controls:
    print(f"  Balanced controls on qubits: {balanced_controls}")

# ── CLASSICAL ──
print("\n" + "-" * 60)
print("  📋 CLASSICAL APPROACH")
print("-" * 60)

start = time.time()
result_c, evals_c, log_c = classical_deutsch_jozsa(n, choice, balanced_controls)
t_classical = time.time() - start

print(f"  Result: {result_c}")
print(f"  Evaluations used: {evals_c}")
print(f"  Worst case for n={n}: {2**(n-1)+1:,}")
print(f"  Time: {t_classical*1000:.3f} ms")

if n <= 5:
    print(f"\n  First evaluations:")
    for x, fx in log_c[:min(8, len(log_c))]:
        print(f"    f({x}) = {fx}")
    if len(log_c) > 8:
        print(f"    ... ({len(log_c) - 8} more)")

# ── QUANTUM ──
print("\n" + "-" * 60)
print("  ⚛️  QUANTUM APPROACH (Deutsch-Jozsa)")
print("-" * 60)

if n <= 15:
    if choice == "constant_0":
        oracle_qc = create_constant_oracle(n, 0)
    elif choice == "constant_1":
        oracle_qc = create_constant_oracle(n, 1)
    else:
        oracle_qc = QuantumCircuit(n + 1, name="Balanced")
        for ctrl in balanced_controls:
            oracle_qc.cx(ctrl, n)

    start = time.time()
    result_q, counts_q, qc = run_deutsch_jozsa(n, oracle_qc, shots=1024)
    t_quantum = time.time() - start

    print(f"  Result: {result_q}")
    print(f"  Oracle queries: 1")
    print(f"  Simulation time: {t_quantum*1000:.3f} ms")
    print(f"  Measurement counts: {dict(sorted(counts_q.items()))}")

    if n <= 6:
        print("\n  Circuit:")
        qc.draw(output='mpl', style='iqp', fold=-1)
        plt.tight_layout()
        plt.show()

        fig2, ax2 = plt.subplots(figsize=(8, 4))
        states = list(counts_q.keys())
        vals = list(counts_q.values())
        colors = ['#1E88E5' if s == '0'*n else '#FF7043' for s in states]
        ax2.bar(states, vals, color=colors, edgecolor='#333', linewidth=1)
        ax2.set_title(f'Measurement Results (n={n}, {choice})', fontweight='bold')
        ax2.set_xlabel('Measured State')
        ax2.set_ylabel('Counts')
        plt.tight_layout()
        plt.show()
else:
    print(f"  (Skipping simulation for n={n} - too large)")
    print(f"  Theoretically: Oracle queries = 1")
    result_q = "constant" if choice.startswith("constant") else "balanced"

# ── SUMMARY ──
print("\n" + "=" * 60)
print("  📊 SUMMARY")
print("=" * 60)
print(f"  {'Metric':<30} {'Classical':>12} {'Quantum':>12}")
print(f"  {'-'*30} {'-'*12} {'-'*12}")
print(f"  {'Oracle queries':<30} {evals_c:>12,} {1:>12}")
```

```
    print(f"  {'Worst case queries':<30} {2**(n-1)+1:>12,} {1:>12}")
    print(f"  {'Speedup factor':<30} {'':>12} {2**(n-1)+1:>11,}x")
    print(f"  {'Result correct?':<30} {'✓':>12} {'✓':>12}")

    # Comparison plot
    plot_complexity_comparison(max_n=max(n, 15))
```

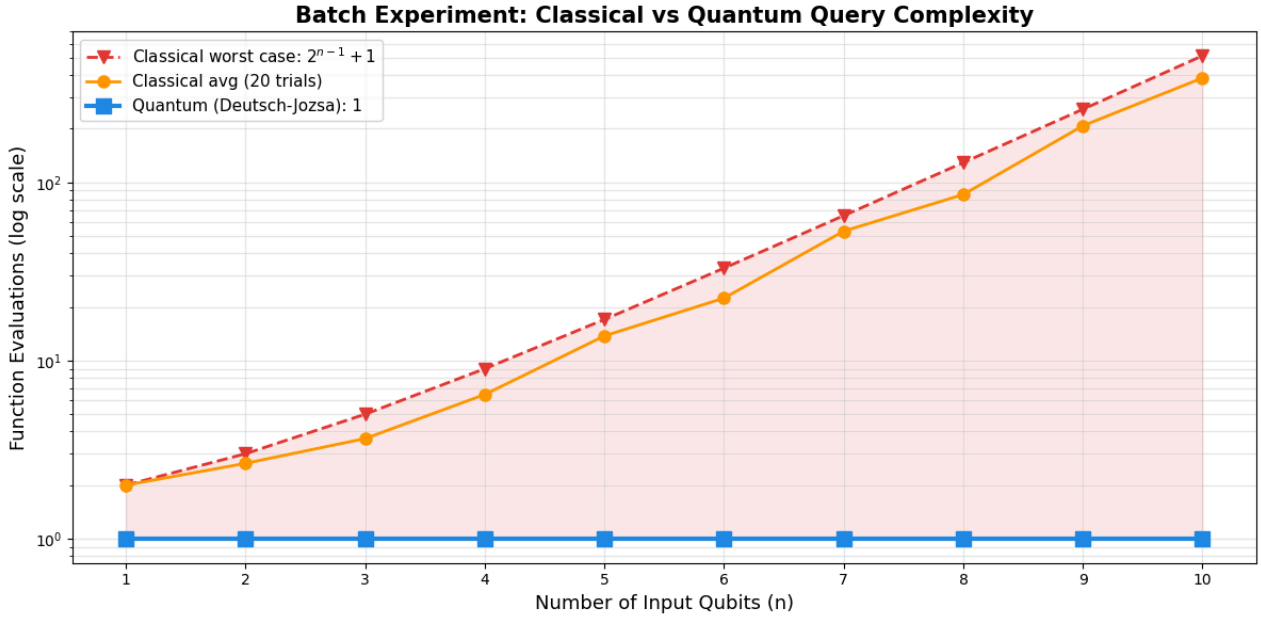
```
N_QUBITS = 4                    # Try: 1, 2, 3, 4, 5, ..., 10
ORACLE_TYPE = "random"           # Options: "constant_0", "constant_1", "balanced", "random"
SEED = 42                        # Change for different random oracles

interactive_comparison(N_QUBITS, oracle_type=ORACLE_TYPE, seed=SEED)
```

```
╔═══════════════════════════════════════════════════════════╗
  ⚛  DEUTSCH-JOZSA: CLASSICAL vs QUANTUM
  Input qubits: n = 4
  Total input space: 2^4 = 16 possible inputs
╚═══════════════════════════════════════════════════════════╝

  Oracle type: balanced
  Balanced controls on qubits: [1, 2]

────────────────────────────────────────────────────────────
  📋 CLASSICAL APPROACH
────────────────────────────────────────────────────────────
  Result: balanced
  Evaluations used: 3
  Worst case for n=4: 9
  Time: 0.014 ms

  First evaluations:
    f(0000) = 0
    f(0001) = 0
    f(0010) = 1

────────────────────────────────────────────────────────────
  ⚛  QUANTUM APPROACH (Deutsch-Jozsa)
```

```
# ============================================================
# CELL 12: Batch Experiment - Multiple Trials
# ============================================================
def batch_experiment(n_range=range(1, 11), trials_per_n=10):
    """Run multiple trials to show avg classical evaluations vs quantum."""
    results = []

    for n in n_range:
        classical_evals_list = []
        quantum_correct = 0

        for trial in range(trials_per_n):
            seed = n * 1000 + trial
            random.seed(seed)

            choice = random.choice(["constant_0", "constant_1", "balanced"])

            if choice in ["constant_0", "constant_1"]:
                balanced_controls = None
            else:
                controls = list(range(n))
                random.shuffle(controls)
                balanced_controls = sorted(controls[:random.randint(1, n)])

            _, evals_c, _ = classical_deutsch_jozsa(n, choice, balanced_controls)
            classical_evals_list.append(evals_c)

            if n <= 12:
                if choice == "constant_0":
                    oqc = create_constant_oracle(n, 0)
                elif choice == "constant_1":
                    oqc = create_constant_oracle(n, 1)
                else:
                    oqc = QuantumCircuit(n + 1)
                    for ctrl in balanced_controls:
                        oqc.cx(ctrl, n)

                rq, _, _ = run_deutsch_jozsa(n, oqc)
                expected = "constant" if choice.startswith("constant") else "balanced"
                if rq == expected:
                    quantum_correct += 1

        avg_c = np.mean(classical_evals_list)
        worst_c = 2 ** (n - 1) + 1
        accuracy = (quantum_correct / trials_per_n) * 100 if n <= 12 else 100.0

        results.append({
            'n': n, 'avg_classical': avg_c,
            'worst_classical': worst_c, 'quantum': 1,
            'quantum_accuracy': accuracy
        })

        print(f"n={n:>2}: Classical avg={avg_c:>10,.1f}, "
              f"worst={worst_c:>10,}, Quantum=1, accuracy={accuracy:.0f}%")

    # Plot
    fig, ax = plt.subplots(figsize=(12, 6))
    ns = [r['n'] for r in results]
    avg_vals = [r['avg_classical'] for r in results]
    worst_vals = [r['worst_classical'] for r in results]
```

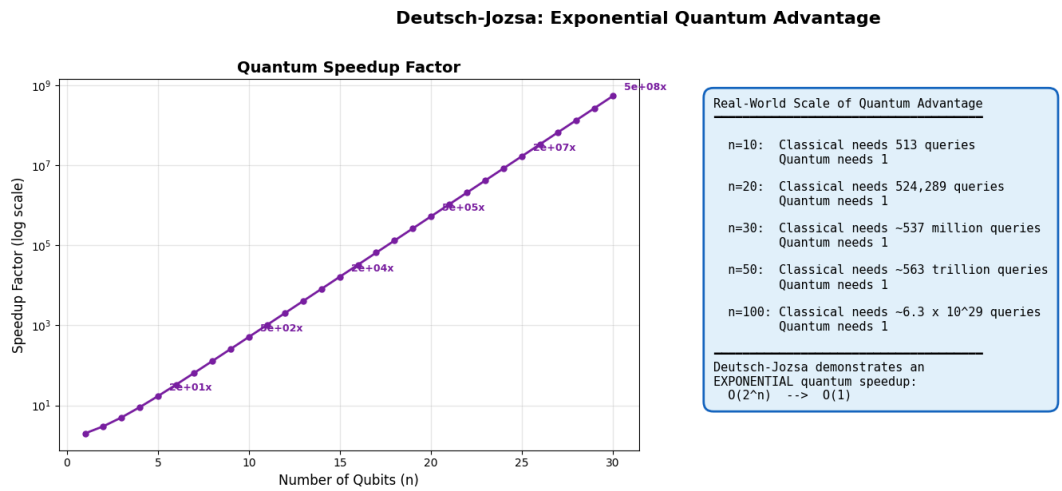
```
    worst_vals = [r['worst_classical'] for r in results]
    q_vals = [r['quantum'] for r in results]

    ax.semilogy(ns, worst_vals, 'v--', color='#E53935', linewidth=2, markersize=8,
                label='Classical worst case: $2^{n-1}+1$')
    ax.semilogy(ns, avg_vals, 'o-', color='#FF9800', linewidth=2, markersize=8,
                label=f'Classical avg ({trials_per_n} trials)')
    ax.semilogy(ns, q_vals, 's-', color='#1E88E5', linewidth=3, markersize=10,
                label='Quantum (Deutsch-Jozsa): 1')
    ax.fill_between(ns, q_vals, worst_vals, alpha=0.1, color='#E53935')
    ax.set_xlabel('Number of Input Qubits (n)', fontsize=13)
    ax.set_ylabel('Function Evaluations (log scale)', fontsize=13)
    ax.set_title('Batch Experiment: Classical vs Quantum Query Complexity',
                 fontsize=15, fontweight='bold')
    ax.legend(fontsize=11, loc='upper left')
    ax.grid(True, alpha=0.3, which='both')
    ax.set_xticks(ns)
    plt.tight_layout()
    plt.show()

    return results


print("Running batch experiment...\n")

batch_results = batch_experiment(n_range=range(1, 11), trials_per_n=20)
```

```
Running batch experiment...

n= 1: Classical avg=       2.0, worst=         2, Quantum=1, accuracy=100%
n= 2: Classical avg=       2.6, worst=         3, Quantum=1, accuracy=100%
n= 3: Classical avg=       3.6, worst=         5, Quantum=1, accuracy=100%
n= 4: Classical avg=       6.5, worst=         9, Quantum=1, accuracy=100%
n= 5: Classical avg=      13.8, worst=        17, Quantum=1, accuracy=100%
n= 6: Classical avg=      22.4, worst=        33, Quantum=1, accuracy=100%
n= 7: Classical avg=      53.2, worst=        65, Quantum=1, accuracy=100%
n= 8: Classical avg=      85.4, worst=       129, Quantum=1, accuracy=100%
n= 9: Classical avg=     207.7, worst=       257, Quantum=1, accuracy=100%
n=10: Classical avg=     385.2, worst=       513, Quantum=1, accuracy=100%
```




```
# ============================================================
# CELL 13: Speedup Analysis
# ============================================================
def speedup_analysis(max_n=30):
    """Show theoretical exponential speedup."""
    n_vals = np.arange(1, max_n + 1)
    classical = 2.0 ** (n_vals - 1) + 1
    quantum = np.ones_like(n_vals, dtype=float)
    speedup = classical / quantum

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
```

```
    ax1.semilogy(n_vals, speedup, 'o-', color='#7B1FA2', linewidth=2, markersize=5)
    ax1.set_xlabel('Number of Qubits (n)', fontsize=12)
    ax1.set_ylabel('Speedup Factor (log scale)', fontsize=12)
    ax1.set_title('Quantum Speedup Factor', fontsize=14, fontweight='bold')
    ax1.grid(True, alpha=0.3, which='both')

    for n_mark in [5, 10, 15, 20, 25, 30]:
        if n_mark <= max_n:
            idx = n_mark - 1
            ax1.annotate(f'{speedup[idx]:.0e}x',
                         xy=(n_vals[idx], speedup[idx]),
                         textcoords="offset points", xytext=(10, 5),
                         fontsize=9, color='#7B1FA2', fontweight='bold')

    # Info panel
    ax2.axis('off')
    text = (
        "Real-World Scale of Quantum Advantage\n"
        "━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━\n\n"
        "  n=10:  Classical needs 513 queries\n"
        "         Quantum needs 1\n\n"
        "  n=20:  Classical needs 524,289 queries\n"
        "         Quantum needs 1\n\n"
        "  n=30:  Classical needs ~537 million queries\n"
        "         Quantum needs 1\n\n"
        "  n=50:  Classical needs ~563 trillion queries\n"
        "         Quantum needs 1\n\n"
        "  n=100: Classical needs ~6.3 x 10^29 queries\n"
        "         Quantum needs 1\n\n"
        "━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━\n"
        "Deutsch-Jozsa demonstrates an\n"
        "EXPONENTIAL quantum speedup:\n"
        "  O(2^n)  -->  O(1)"
    )
    ax2.text(0.05, 0.95, text, transform=ax2.transAxes,
             fontsize=11, verticalalignment='top', fontfamily='monospace',
             bbox=dict(boxstyle='round,pad=0.8', facecolor='#E3F2FD',
                       edgecolor='#1565C0', linewidth=2))

    plt.suptitle('Deutsch-Jozsa: Exponential Quantum Advantage',
                 fontsize=16, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.show()


speedup_analysis(max_n=30)
```




```
# ============================================================
# CELL 14: Step-by-Step Walkthrough
# ============================================================
def step_by_step_walkthrough(n=2):
    """Build the circuit step by step with statevector snapshots."""
    print(f"\n{'='*60}")
    print(f"  STEP-BY-STEP WALKTHROUGH (n={n})")
```

```
print(f"{'='*60}")

qc = QuantumCircuit(n + 1)

# Step 0
sv = Statevector.from_instruction(qc)
print(f"\n Step 0: Initial state |{'0'*n}>|0>")
print(f"   Statevector: {sv}")

# Step 1
qc.x(n)
sv = Statevector.from_instruction(qc)
print(f"\n Step 1: Apply X to output qubit -> |{'0'*n}>|1>")
print(f"   Statevector: {sv}")

# Step 2
qc.h(range(n + 1))
sv = Statevector.from_instruction(qc)
print(f"\n Step 2: Apply H to all qubits -> superposition")
print(f"   Creates equal superposition of all 2^{n} = {2**n} input states")
print(f"   Statevector (first 8): {np.round(sv.data[:8], 4)}")

# Step 3
qc.cx(0, n)
sv = Statevector.from_instruction(qc)
print(f"\n Step 3: Apply balanced oracle (CNOT from qubit 0)")
print(f"   Phase kickback encodes function info into phases!")
print(f"   Statevector (first 8): {np.round(sv.data[:8], 4)}")

# Step 4
qc.h(range(n))
sv = Statevector.from_instruction(qc)
print(f"\n Step 4: Apply H to input qubits -> interference")
print(f"   Constructive/destructive interference reveals function type")
print(f"   Statevector (first 8): {np.round(sv.data[:8], 4)}")

# Probabilities
# Qiskit uses LITTLE-ENDIAN ordering: qubit 0 is the LEAST significant bit.
# For a circuit with qubits [q0, q1, ..., qn], statevector index i
# corresponds to |qn ... q1 q0> where q0 is the rightmost bit.
# Our circuit: qubits 0..n-1 are input, qubit n is output (ancilla).
#
# To read states properly:
#   - bit position 0 (LSB) = qubit 0 (input)
#   - bit position n (MSB) = qubit n (output/ancilla)

probs = sv.probabilities()
total_qubits = n + 1
print(f"\n Step 5: Measurement probabilities")
print(f"   (Qiskit little-endian: qubit 0 = rightmost bit)")
print()

# Accumulate probability that all INPUT qubits are 0
zero_input_prob = 0.0

for i, p in enumerate(probs):
    if p > 1e-10:
        # Qiskit ordering: binary string is |q_n q_{n-1} ... q_1 q_0>
        bitstring = format(i, f'0{total_qubits}b')  # MSB=qubit n, LSB=qubit 0

        # Extract input qubits (q0..q_{n-1}) = rightmost n bits
        input_bits = bitstring[1:]    # last n chars (qubits n-1 down to 0)
        output_bit = bitstring[0]     # first char (qubit n = ancilla)

        # For display: show input in conventional order q_{n-1}...q_0
        print(f"     |input={input_bits}> |ancilla={output_bit}> : {p:.4f}")

        # Check if all input qubits are 0
        if all(b == '0' for b in input_bits):
            zero_input_prob += p

print(f"\n   P(all input qubits = |{'0'*n}>) = {zero_input_prob:.4f}")
if zero_input_prob < 0.01:
    print(f"   -> Input qubits are NOT all zero -> BALANCED ✓")
    print(f"   (Quantum algorithm correctly identified the balanced function!)")
else:
    print(f"   -> Input qubits are all zero -> CONSTANT ✓")
    print(f"   (Quantum algorithm correctly identified the constant function!)")

# Also verify by running the actual circuit with measurement
print(f"\n Verification: Running the circuit with 1024 shots...")
qc_run = QuantumCircuit(n + 1, n)
```